

Terminé

1) Quelques questions :

La visibilité par défaut d'une entité appartenant à une classe est :

- A) public
- B) private
- C) protected

Quelle est la bonne proposition pour inclure un fichier header écrit par un utilisateur ?

- A) #include "file.h"
- B) #include <file.h>
- C) #include <file>
- D) #include file.h

Choisissez l'utilisation de l'opérateur delete adapté à l'expression 'ptr=new int[100]'.

- A) delete ptr;
- B) delete ptr[];
- C) delete[] ptr;
- D) []delete ptr;

Choisissez la bonne définition d'une fonction virtuelle pure :

- A) virtual void f()=0 { }
- B) void virtual f()=0 { }
- C) virtual void f() {} = 0;
- D) aucune

Quel est l'utilité de try{} catch(){} ?

- A) Tenter de réaliser un calcul très compliqué
- B) Tenter de réaliser une opération qui peut échouer
- C) Tenter de lancer un programme extérieur et recevoir sa réponse

2) Donner la signification des différents const

- const Type MAX = 10;

- const Type aClass::f();

- void aClass::f(const Type& farg);

- void aClass::f() const

Jean-Marc Perronne -2019/2020

3) Ecrire la structure de donnée C++ pour laquelle tout est public nommée *CurveValue* permettant le fragment de code suivant :

```
CurveValue v1;
CurveValue v2(5,10);

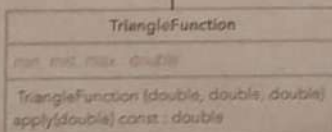
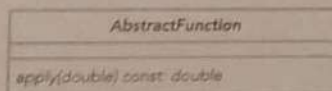
std::cout << "v2(" << v2.x << ", " << v2.y << ")";
std::cout << std::endl;
v1.printOn(std::cout);
std::cout << std::endl;
v2.printOn(std::cout);
std::cout << std::endl;
std::cout << std::endl;

v1=v2;
v2.printOn(std::cout);
```



```
v2(5, 10)
(0, 0)
(5, 10)
```

4) En fonction du diagramme de classes ci-dessous et du cas d'utilisation proposé, écrivez les classes *AbstractFunction* et *TriangleFunction*.

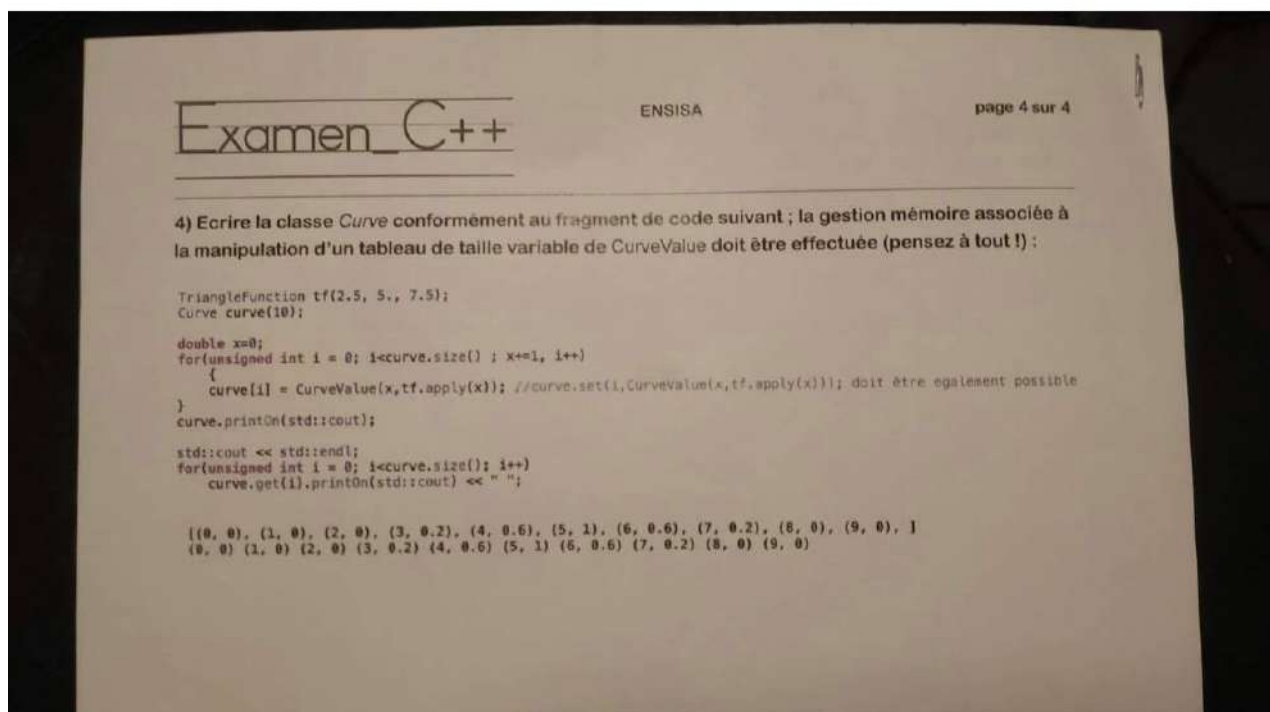


```
TriangleFunction tf(2.5, 5., 7.5);
double x=0;
for(unsigned int i = 0; i<10 ; x+=1, i++)
{
    CurveValue(x,tf.apply(x)).printOn(std::cout) << ' ';
}
```



```
(0, 0) (1, 0) (2, 0) (3, 0.2) (4, 0.6) (5, 1) (6, 0.6) (7, 0.2) (8, 0) (9, 0)
```

Terminé



Plus

Nom :

Ecrire les classes nécessaires à la modélisation d'une Balise (Tag) XML.

Un Tag est une structure arborescente.

Un Tag possède un nom, peut avoir plusieurs arguments, peut disposer d'une valeur et présenter plusieurs "sous Tag".

Par exemple :

```
<tag1 a1= "val1" a2= "val2"> <tag11>tag11val</tag11> <tag12>tag12val</tag12> </tag1>
```

Le Tag de nom tag1 possède:

- les arguments a1 de valeur val1 et a2 de valeur val2,
- le "sous Tag" de nom tag11, de valeur valTag11 et le "sous Tag" de nom tag12 de valeur valTag12. Ces deux Tags ne présentent ni arguments ni "sous Tags"

Le diagramme de classes fourni vous donne essentiellement la structure de Tag.

La classe LinkedElt est abstraite et la méthode toString() est virtuelle pure.

Le diagramme inspiré d'un diagramme d'objets donne une configuration d'instances de Tag conforme à l'exemple.

Un scénario d'exécution vous indique les méthodes strictement indispensables pour les différentes classes ainsi que les comportements souhaités.

Diagramme de classes

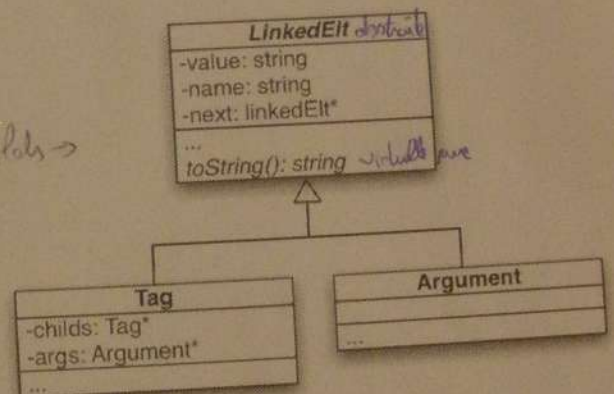
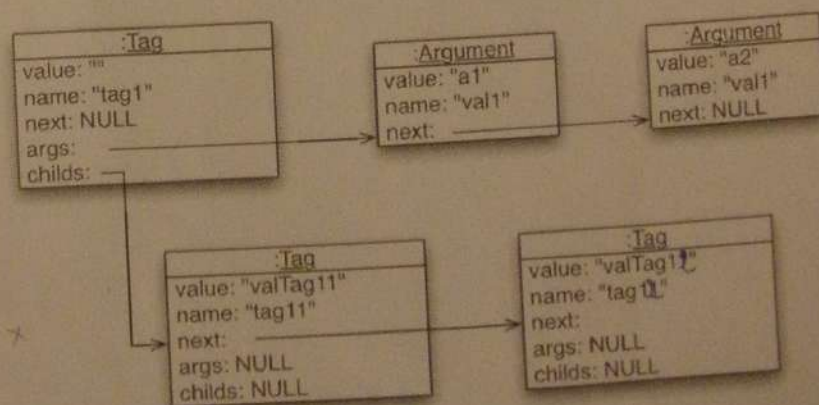


Diagramme d'objets



Console

```
<tag1></tag1>
a1= "val1"
<tag1 a1= "val1" a2= "val2"><tag11>valTag11</tag11><tag12>valTag12</tag12></tag1>
<tag12>valTag12</tag12>
valTag11
```

LinkedListElement.h

1

```
#ifndef LINKED_ELT
#define LINKED_ELT
```

```
class LinkedListElement
```

```
{ private:
    string value;
    string name;
    LinkedListElement next;
```

```
public:
    virtual void toString() const;
    LinkedListElement();
```

```
#endif
```

LinkedListElement.cpp

```
#include "LinkedListElement.h"
```

```
LinkedListElement::LinkedListElement() {}
```

```
void LinkedListElement::ToString() {}
```

Tag.h

```
#include "Argument.h"
```

```
#ifndef TAG
```

```
#define TAG
```

```
class Tag : public LinkedListElement
```

```
{ private:
    vector<Tag> children;
    vector<Argument> args;
```

```
public:
```

```
Tag();
```

```
void setName(string s);
```

```
void setValue(string v);
```

```
void setArgument(Argument a);
```

```
void setChild(Tag t);
```

```
string getTagValue(string v);
```

```
virtual void toString();
```


Tag.cpp

#include "Tag.h"

Tag::Tag()

{ This.value = " ";

This.name = " ";

This.next = null;

This.childs = null;

This.args = null; }

void Tag::setname(string s) {
 this.name = s; }

void Tag::setArgument(argument a) {
 if (args == null)
 args = a; else args->next = a; }

void Tag::setValue(string v) {
 this.value = v; }

~~void Tag::setChild~~

void Tag::setChild(Tag t) {
 if (this.childs == null)
 childs = t;

else

childs->next = t; }

void Tag::toString() {

cout << this.name << this.args->toString() <<

this.childs->toString() << " " << this.name << " "; }

~~void Tag::toString() {~~

~~cout << this.name << this.args->toString() <<~~

Argument.h

```
#include "Tag.h"
#include ARGUMENT
#define _____

class Argument : public LinkedLT
{
public:
    Argument();
    virtual void toString();
    Argument(string a, string b);
    virtual void toString();
}
```

Argument.cpp

```
#include "Argument.h"
Argument::Argument()
{
    this->value = "";
    this->name = "";
    this->next = null;
}

Argument::Argument(string a, string b)
{
    this->name = a;
    this->value = b;
}
```

```
void Argument::toString()
{
    cout << this->name << "=" << this->value << this->next->toString();
}
```